Vel Tech Rangarajan Dr. Sagunthala R&D Institute of Science and Technology School of Computing

Department of Computer Science and Engineering

10211CS224 / FULL STACK APPLICATION DEVELOPMENT (JAVA)

PROJECT REVIEW : MILESTONE PROJECT – 1

Date: 27-03-2026

PROJECT TITLE:REAL-TIME SYNC ENGINE

SDG: SDG 09- Industry and Innovation

Presented By:

D.Sri Venkata Siddardha– VTU27240 –
Reg.No:23UECS0712 **Courses Handling Faculty:**

Dr.Hafizur Rahman -Assistant Professor

1
2
3
4
5
6
7
8
9

INTRODUCTION

- **Background of the Problem:** Traditional web apps require page refresh for updates, causing delay and poor user experience in real-time communication.
- **Why this Project is Important:** Modern applications demand instant data sharing (chat, collaboration), which cannot be handled efficiently using basic HTTP requests.
- **Real-World Relevance:** Used in apps like WhatsApp, Discord, and live collaboration tools where users expect instant message delivery and synchronization.
- **Brief Overview of the System:** A real-time chat and event sync system using Node.js, Express, and Socket.IO where users join rooms, send messages, and receive instant updates without refreshing the page.

PROBLEM STATEMENT

1
2
3
4
5
6
7
8
9

- Example Statement:

Most traditional web applications rely on HTTP requests, which require users to refresh the page to see updates, leading to delays and a poor user experience. This becomes a major issue in realtime communication, where instant interaction is expected. Existing basic systems fail to efficiently synchronize data among multiple users, resulting in message delays, inconsistent information, and lack of live updates. These limitations make collaboration and communication ineffective, especially in multi-user environments. Therefore, there is a clear need for a system that enables instant data exchange and real-time synchronization without requiring page refresh.

WebSocket is a communication protocol that provides a persistent, full-duplex (two-way) connection between a client and server over a single TCP connection. Unlike HTTP, which follows a request-response model, WebSocket allows both the client and server to send messages to each other at any time without needing to re-establish a connection.

PROPOSED SYSTEM

1 2 3 4 5 6 7 8 9

Description of your solution:

- Developed a real-time multi-user chat and event synchronization system using Node.js, Express, and Socket.IO.
- Uses Web-Sockets to enable instant communication without page refresh.
- Allows users to join named rooms for organized and isolated interactions.

How it solves existing issues :

- Eliminates page refresh by using Web-Sockets for continuous real-time connection.
- Ensures data consistency by synchronizing updates across all users in a room.
- Handles multiple users efficiently without performance drop in communication.
- Provides instant message history to new-users, avoiding missing information.
- Improves user experience with seamless and uninterrupted interaction.

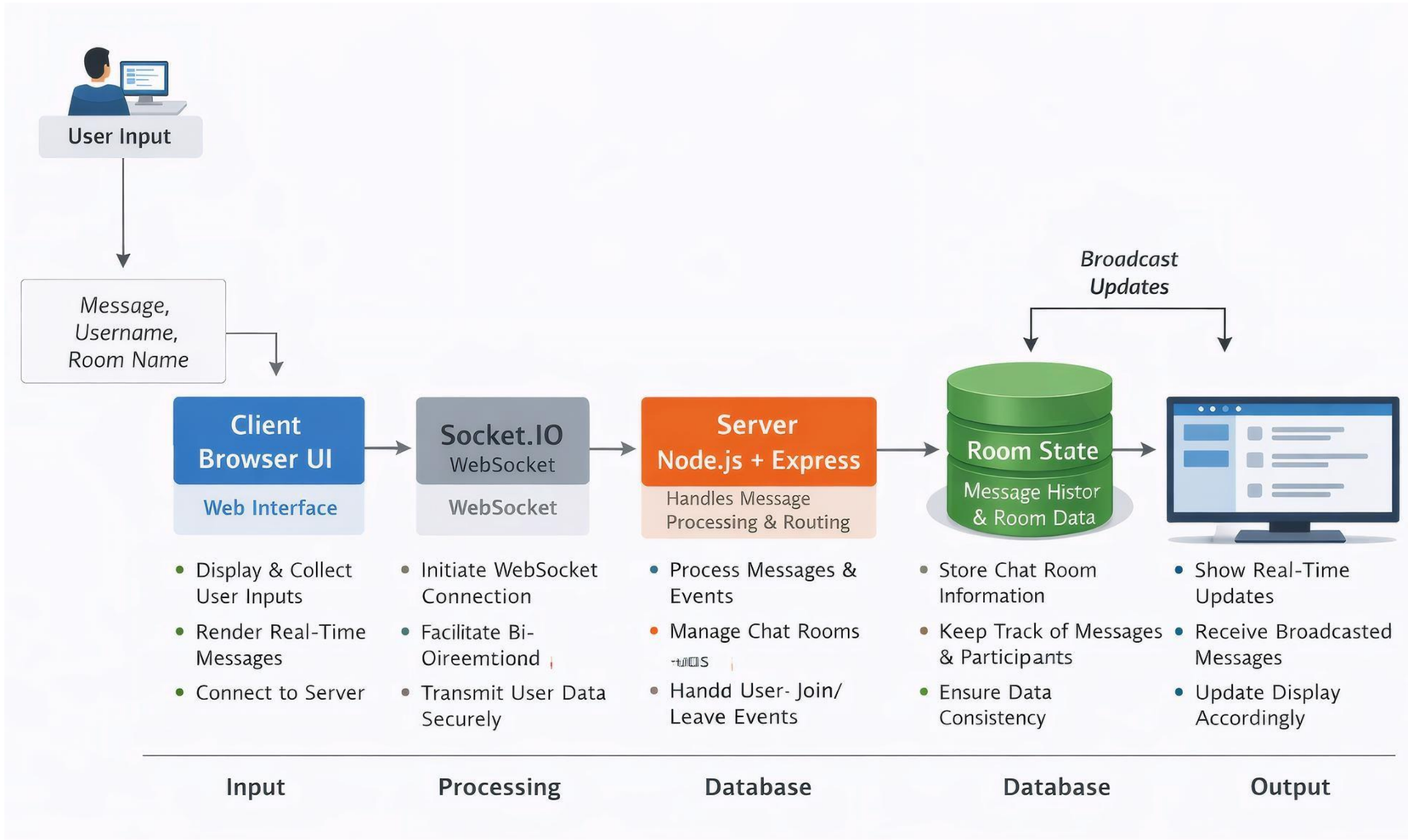
Advantages over current systems:

- Provides real-time communication instead of delayed updates in traditional systems.
- Eliminates the need for repeated HTTP requests, reducing server load.
- Ensures instant data synchronization across multiple users.

- Delivers faster response time compared to polling-based systems.
- Offers a smoother and more interactive user experience.

PROPOSED SYSTEM ARCHITECTURE

- **block diagram:**


User Input
Message, Username, Room Name
Client Browser UI
Web Interface
Socket.IO
WebSocket
WebSocket
Server
Node.js + Express
Handles Message Processing & Routing
Broadcast Updates
Room State
Message Histor & Room Data
Display & Collect User Inputs
Render Real-Time Messages
Connect to Server
Initiate WebSocket Connection
Facilitate Bi-Oireemtiond
Transmit User Data Securely
Process Messages & Events
Manage Chat Rooms
Handd User- Join/ Leave Events
Store Chat Room Information
Keep Track of Messages & Participants
Ensure Data Consistency
Show Real-Time Updates
Receive Broadcasted Messages
Update Display Accordingly
Input
Processing
Database
Database
Output

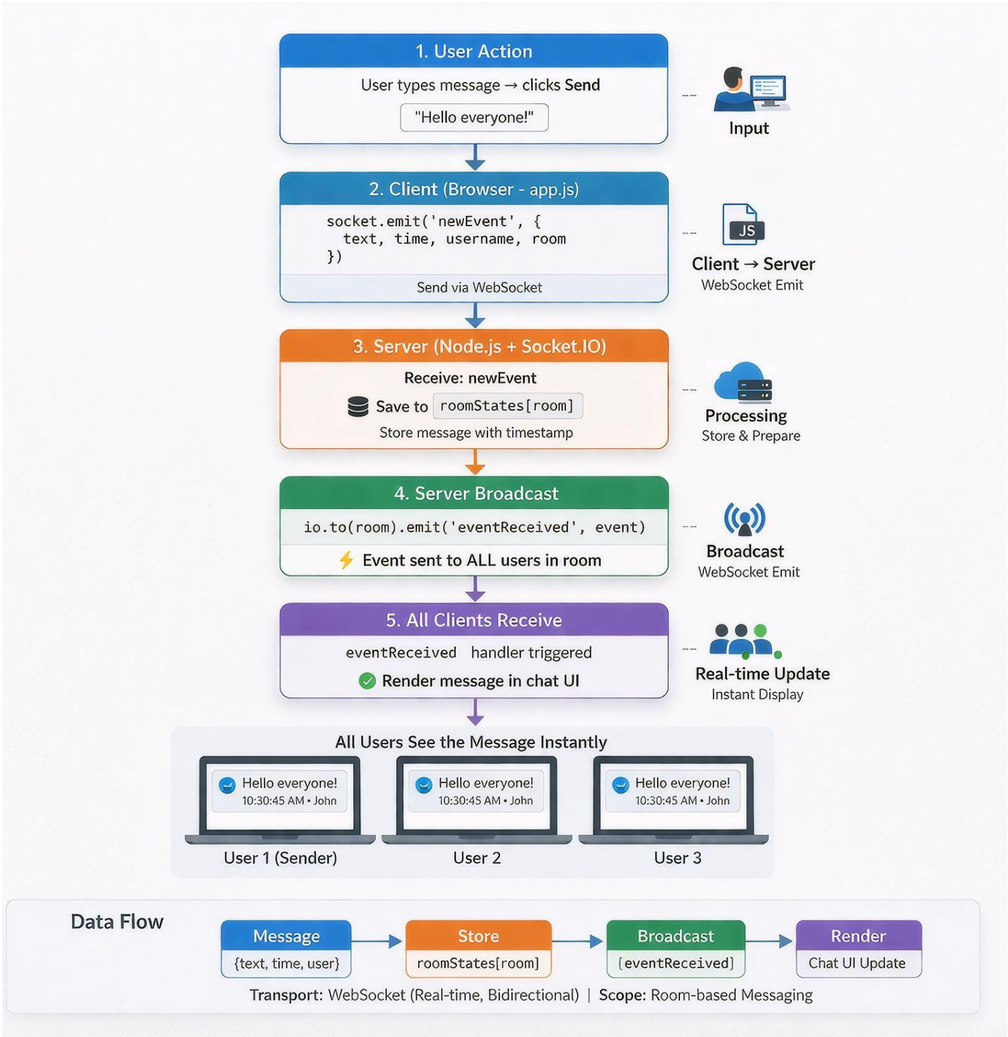
PROPOSED SYSTEM ARCHITECTURE

1
2
3
4
5
5
6
7
8
9

- **Input**: User enters message, username, and room name through the web interface.
- **Processing:** Client sends data via Socket.IO → Server processes it, stores it in room state, and broadcasts to all users in that room.
- **Output:** All connected users receive and see messages instantly on their screen without page refresh.

WORKFLOW

Dataflow Diagram:



TECHNOLOGIES USED

1
2
3
4
5
6
7
8

- **Frontend**: HTML, CSS, JavaScript (Client-side UI, handles user interaction and Socket.IO client)
- **Backend:** Node.js with Express, Socket.IO (handles real-time communication and event processing)
- **Database:** In-memory storage (roomStates object) for messages and room data *(can be extended to MongoDB if needed)*
- **Tools:** VS Code, Nodemon, Git/GitHub, Browser (Chrome)
- **Hardware/Software Requirements:** System with minimum 4GB RAM, Node.js (v16+), modern web browser, internet/local network connectivity

MODULE DESCRIPTION

- **Module 1: User & Room Management**
  Manages user entry into the system by collecting username and room name. It handles creation and joining of chat rooms, ensuring users are grouped correctly. It also tracks active users in each room and triggers join/leave notifications to maintain session awareness.
- **Module 2: Real-Time Communication**
  Implements bidirectional communication using Socket.IO over WebSockets. It establishes a persistent connection between client and server, allowing instant transmission of messages without HTTP requests. This module ensures low latency and continuous data flow.
- **Module 3: Event Processing & Synchronization**
  Handles incoming events (newEvent) from clients, processes message data (text, time, username), and stores it in the respective room state. It then broadcasts the processed event (eventReceived) to all users in the room, ensuring real-time synchronization and consistency across clients.
- **Module 4: Data Storage & Message History**

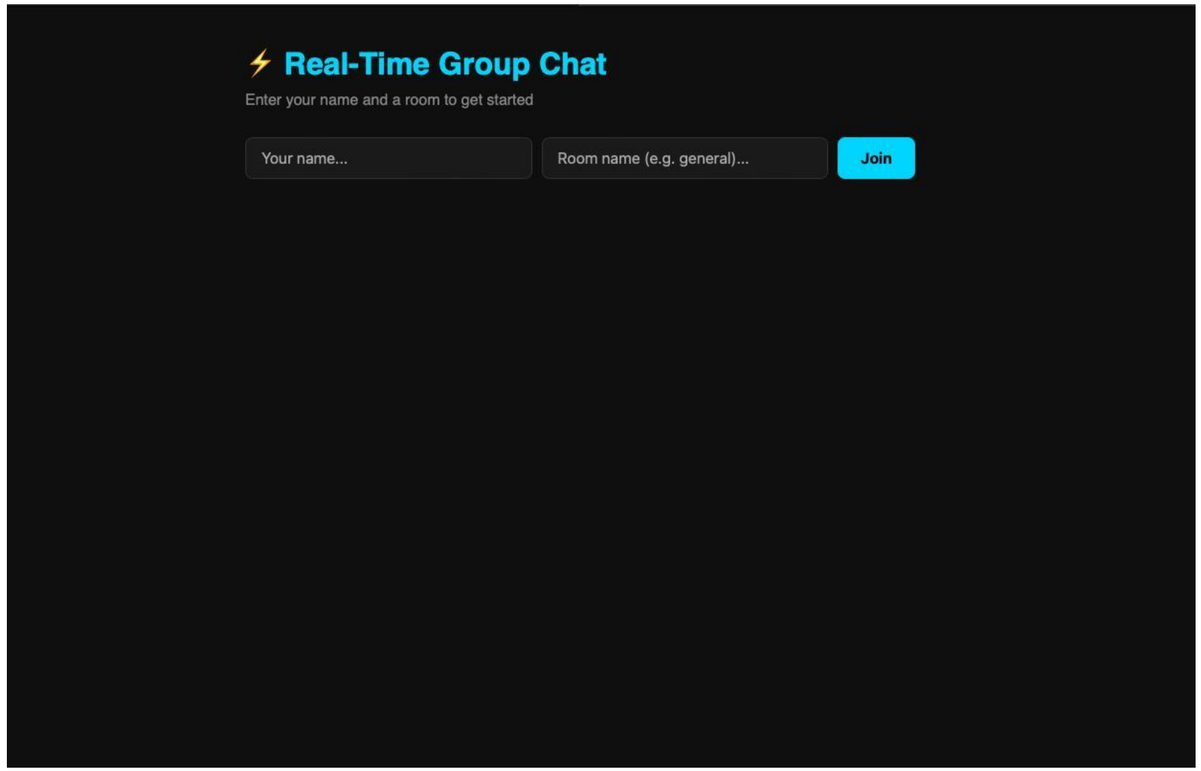
IMPLEMENTATION SCREENSHOTS

Maintains room-wise message storage using an in-memory data structure (e.g., roomStates). It allows retrieval of previous messages when a new user joins, ensuring no data loss during a session.

This module can be extended to use databases like MongoDB for persistent storage.

- HOME PAGE

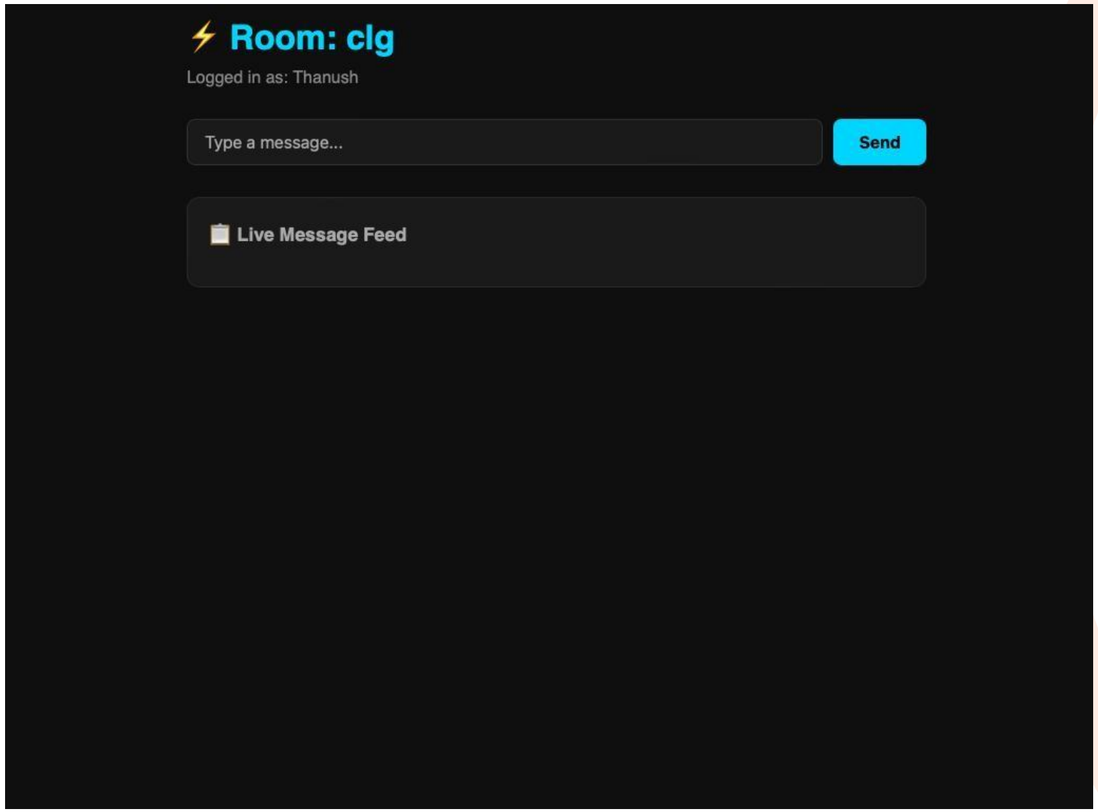
IMPLEMENTATION SCREENSHOTS



In the home page we see that ,the user can enter his name and the room number or id for the joining the room and chart with the ohter persons present in the same room

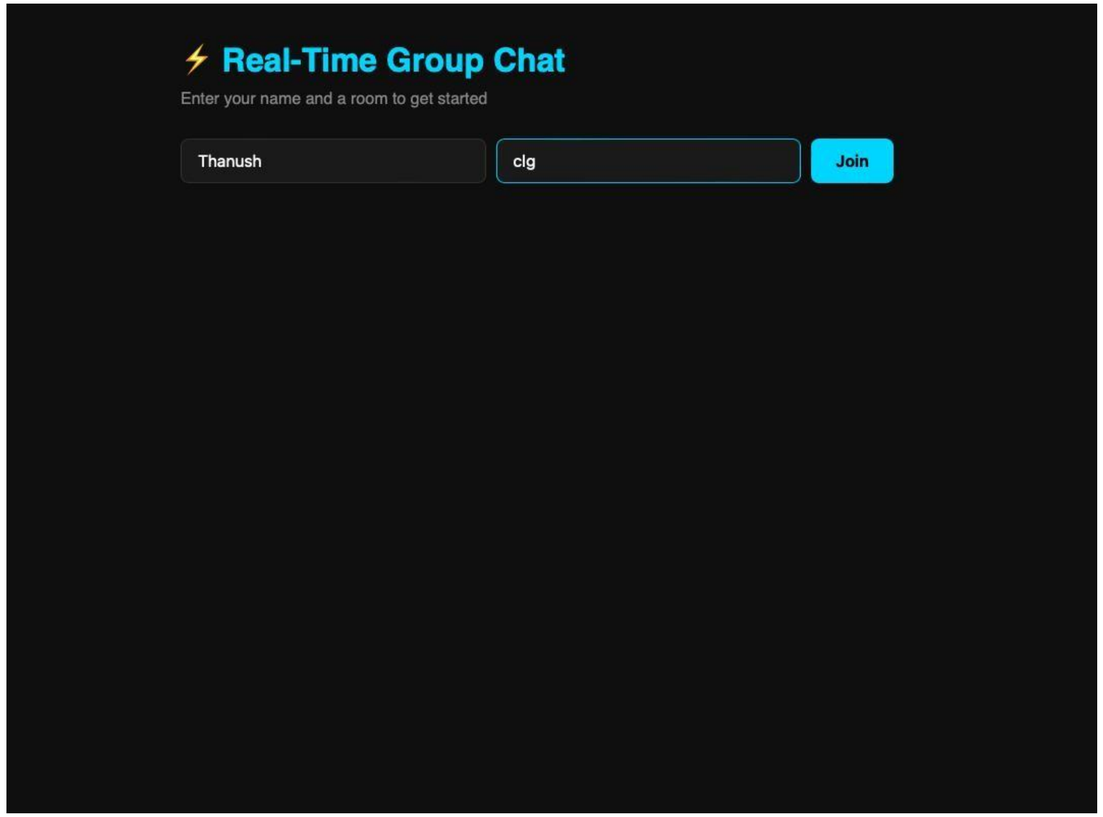
IMPLEMENTATION SCREENSHOTS

- Main module:



IMPLEMENTATION SCREENSHOTS

1
2
3
4
5
6
7
8
9



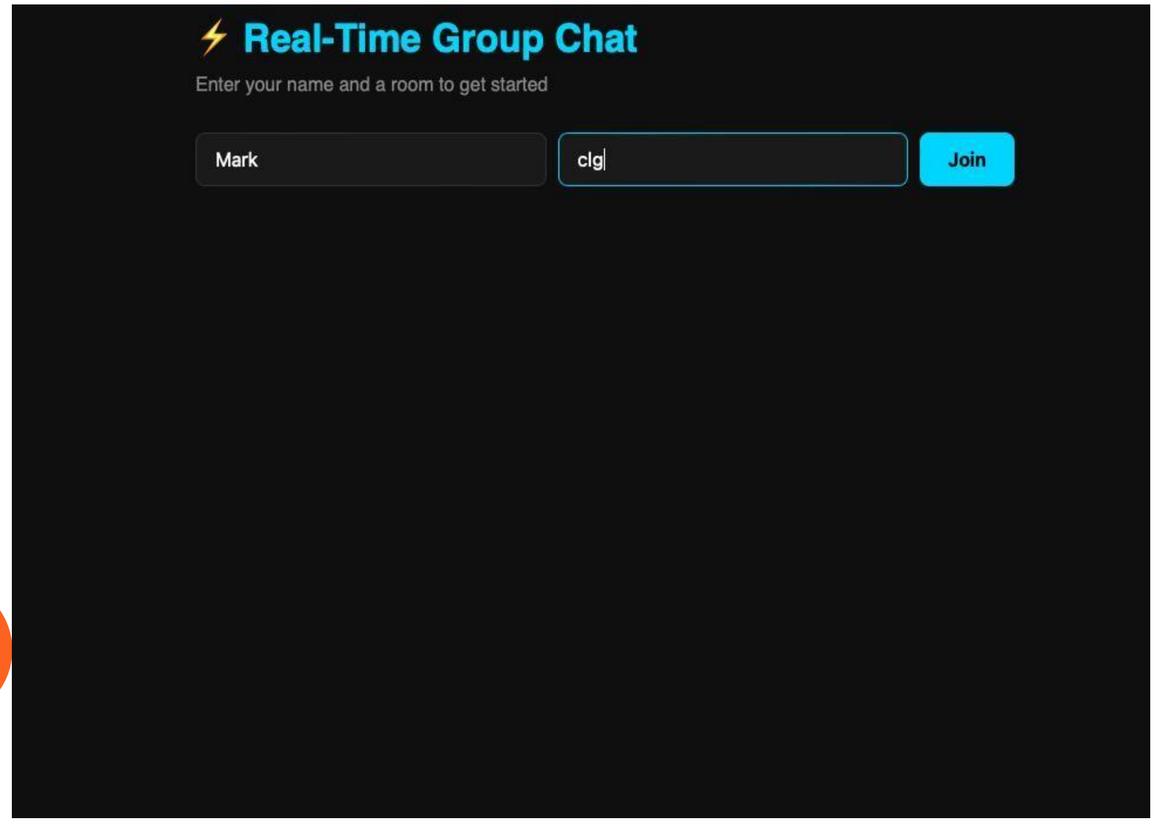
IMPLEMENTATION SCREENSHOTS

Here you can see that the User can enter the Username and the Room no and login into the room

IMPLEMENTATION SCREENSHOTS

1
2
3
4
5
6
7
8(A)

- Main module:

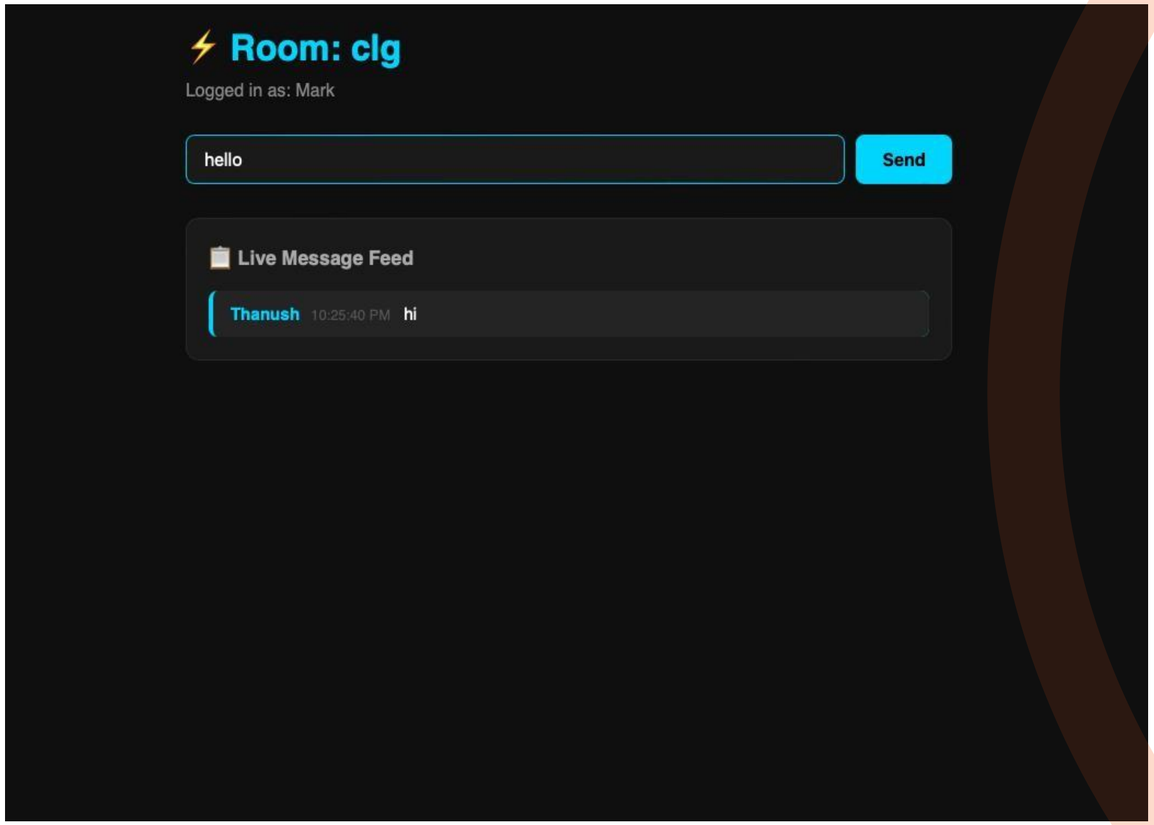


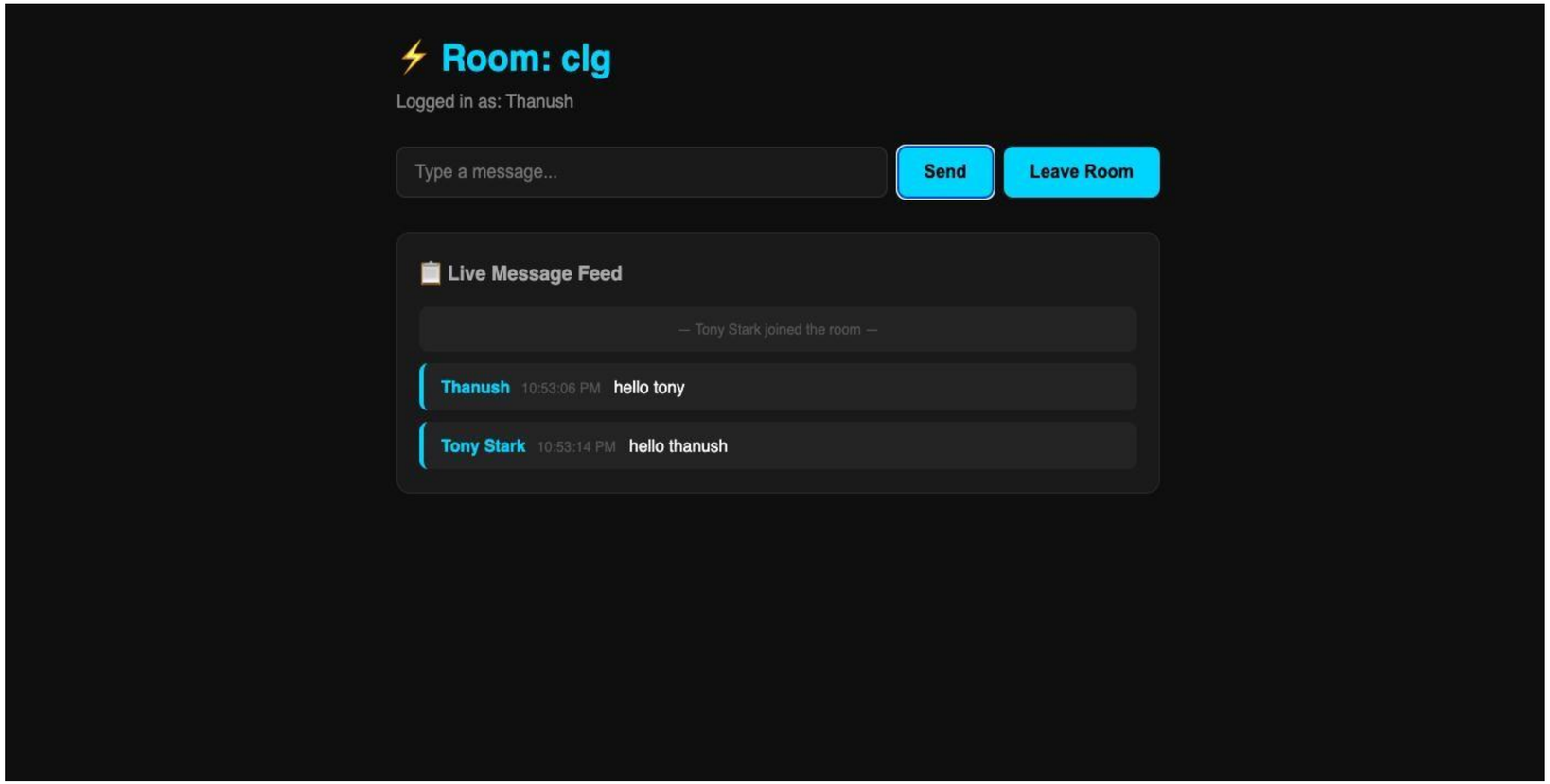
IMPLEMENTATION SCREENSHOTS

1
2
3
4
5
6
7
8(B)

Here another person trying to login with his name and the same no number ,so that he can join the room and start communicate with the users present in the room.

- **Output:**





So the final output is both persons add in the room no:clg and start charting with ohters.

ADVANTAGES & LIMITATIONS

Advantages:

IMPLEMENTATION SCREENSHOTS

- Enables real-time communication with zero page refresh using WebSockets.
- Faster data transfer compared to traditional HTTP request–response systems.
- Supports multiple users and room-based interaction efficiently.
- Provides instant synchronization, ensuring all users see the same data.
- Simple and lightweight architecture, easy to develop and deploy.
- Scalable for small to medium applications with minimal configuration.

Limitations:

•Uses in-memory storage, so data is lost when the server restarts.

•Not suitable for large-scale systems without proper database integration.

•Lacks advanced features like authentication, encryption, and security controls.

•Performance may degrade with a very high number of concurrent users.

•Depends heavily on stable internet/WebSocket connection.

•No built-in fault tolerance or backup mechanism.

Thank you